FILE: C:\HCL_TRAINING\server\src\server.js

LINES: 110 | SIZE: 3.6 KB | LANGUAGE: JS

MODULE: Express Server Initialization, Middleware, Healthcheck, and Auto-Seed

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const path = require('path');
const { connectDB } = require('./config/db');
const errorHandler = require('./middleware/errorHandler');

// Route imports
const authRoutes = require('./routes/authRoutes');
const dashboardRoutes = require('./routes/dashboardRoutes');
const jobRoutes = require('./routes/jobRoutes');
const resumeRoutes = require('./routes/resumeRoutes');
const candidateRoutes = require('./routes/candidateRoutes');
const screeningRoutes = require('./routes/screeningRoutes');
const pipelineRoutes = require('./routes/pipelineRoutes');
const searchRoutes = require('./routes/searchRoutes');
const interviewRoutes = require('./routes/interviewRoutes');
const reportRoutes = require('./routes/reportRoutes');
const notificationRoutes = require('./routes/notificationRoutes');
const settingsRoutes = require('./routes/settingsRoutes');

const app = express();

// Middleware
app.use(express.json({ limit: '20mb' }));
app.use(express.urlencoded({ extended: true, limit: '20mb' }));
app.use(cookieParser());

// CORS Configuration
const allowedOrigin = process.env.CLIENT_URL || 'http://localhost:5173';
app.use(
  cors({
    origin: function (origin, callback) {
      // Allow requests with no origin (like mobile apps, curl, or same-origin)
      if (!origin || origin === allowedOrigin || origin.includes('localhost') || origin.
> includes('127.0.0.1')) {
        callback(null, true);
      } else {
        callback(null, true); // Dev-friendly permissive CORS
      }
    },
    credentials: true,
  })
);

// Serve static uploads
app.use('/uploads', express.static(path.join(__dirname, '../uploads')));

// Health check endpoint
app.get('/api/health', (req, res) => {
  res.status(200).json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    service: 'HireSense AI Recruitment Engine',
    version: '1.0.0',
  });
});

// Mount Routes
app.use('/api/auth', authRoutes);
app.use('/api/dashboard', dashboardRoutes);
app.use('/api/jobs', jobRoutes);
app.use('/api/resumes', resumeRoutes);
app.use('/api/candidates', candidateRoutes);
app.use('/api/screening', screeningRoutes);
```

```
66 |  app.use('/api/pipeline', pipelineRoutes);
67 |  app.use('/api/search', searchRoutes);
68 |  app.use('/api/interviews', interviewRoutes);
69 |  app.use('/api/reports', reportRoutes);
70 |  app.use('/api/notifications', notificationRoutes);
71 |  app.use('/api/settings', settingsRoutes);
72 |
73 |  // Error Handling Middleware
74 |  app.use(errorHandler);
75 |
76 |  const PORT = process.env.PORT || 5000;
77 |
78 |  // Start Server and Connect DB
79 |  async function startServer() {
80 |    try {
81 |      await connectDB();
82 |
83 |      // Check if database is empty; if so, automatically run seed script!
84 |      const Job = require('./models/Job');
85 |      const existingJobs = await Job.countDocuments();
86 |      if (existingJobs === 0) {
87 |        console.log('Detected empty database on startup. Running initial seed...');
88 |        const { seedDatabase } = require('./utils/seedData');
89 |        await seedDatabase();
90 |      }
91 |
92 |      app.listen(PORT, () => {
93 |        console.log(`=======================================================`);
94 |        console.log(`  HireSense AI Server running on port ${PORT}`);
95 |        console.log(`  Environment: ${process.env.NODE_ENV || 'development'}`);
96 |        console.log(`  Healthcheck: http://localhost:${PORT}/api/health`);
97 |        console.log(`=======================================================`);
98 |      });
99 |    } catch (error) {
100 |      console.error('Fatal server startup failure:', error);
101 |      process.exit(1);
102 |    }
103 |  }
104 |
105 |  if (process.env.NODE_ENV !== 'test') {
106 |    startServer();
107 |  }
108 |
109 |  module.exports = app;
110 |
```